

RELATÓRIO DE SOLUÇÕES — BLOQUEADOR DE INSERT EM COTACOES_FRETE

Briefing técnico para correção de falha crítica no banco de dados e persistência de estado

08 de agosto de 2026

1. INFORMAÇÕES GERAIS E PRIORIDADE

Este documento detalha a correção para a falha crítica de **INSERT** no banco de dados que impede a criação de novas solicitações de frete. O sistema apresenta regressões em funcionalidades de busca e perda de estado ao alternar abas no navegador. A prioridade é **CRÍTICA**, pois o fluxo principal de negócio está interrompido.

Checkpoint atual: d0de9721

ORM: Drizzle ORM (NÃO utilizar sintaxe Prisma)

Status: Bloqueado

2. CAUSA RAIZ IDENTIFICADA

2.1 Análise do Erro SQL

O erro *"Failed query: insert into cotacoes_frete..."* ocorre porque o Drizzle ORM está tentando inserir 31 campos simultaneamente, incluindo campos que chegam como **undefined** do frontend. O SQL gerado contém placeholders (?) sem valores correspondentes, resultando em falha na execução da query.

2.2 Conflitos de Schema e Payload

Identificamos três inconsistências estruturais:

- Campos Inexistentes:** O frontend envia `pedidoCnpj`, `pedidoEndereco` e `pedidoCep`, que não constam no schema da tabela.
- Mapeamento de Dimensões:** O frontend envia um JSON string em `dimensoes`, mas o banco espera três colunas decimais separadas (`dimensoesLargura`, `dimensoesAltura`, `dimensoesComprimento`).
- Tipagem de Peso:** O peso é enviado como string ou 0, causando falha na validação de tipos numéricos do banco.

3. SOLUÇÃO TÉCNICA: BACKEND (server/routers/logistica.ts)

Substituir a lógica de criação pelo código abaixo para garantir que apenas campos válidos sejam enviados ao banco:

```
.create(async ({ input }) => {  
  let dimensoesLargura: number | null = null;<br/>  
  let dimensoesAltura: number | null = null;<br/>  
  let dimensoesComprimento: number | null = null;<br/>  
  let pesoKg: number | null = null;  
  
  if (input.dimensoes) {  
    try {  
      const volumes = JSON.parse(input.dimensoes);  
      if (Array.isArray(volumes) && volumes.length > 0) {  
        const v = volumes[0];  
        dimensoesLargura = Number(v.largura) || null;  
        dimensoesAltura = Number(v.altura) || null;  
        dimensoesComprimento = Number(v.comprimento) || null;  
        pesoKg = volumes.reduce((acc: number, vol: any) => acc + (Number(vol.peso) || 0))  
      }  
    } catch (e) {  
      console.error('Erro ao parsear dimensoes:', e);  
    }  
  }  
  
  if (!pesoKg && input.pesoKg) pesoKg = parseFloat(input.pesoKg);  
  const cnpjLimp0 = input.destinatarioCnpj ? input.destinatarioCnpj.replace(/\D/g, '') :  
  
  const insertData: Record<string, any> = {<br/>  
    destinatarioNome: input.destinatarioNome,<br/>  
    municipio: input.municipio,<br/>  
    estado: input.estado,<br/>  
    status: 'fila',<br/>  
    temRetrabalho: false,  
  };  
  
  if (input.solicitanteNome) insertData.solicitanteNome = input.solicitanteNome;  
  if (cnpjLimp0) insertData.destinatarioCnpj = cnpjLimp0;  
  if (input.cepDestino) insertData.cepDestino = input.cepDestino;  
  if (pesoKg && pesoKg > 0) insertData.pesoKg = pesoKg;  
  if (input.valorNf) insertData.valorNf = parseFloat(input.valorNf) || 0;  
  if (input.observacoes) insertData.observacoes = input.observacoes;  
  if (input.tipoMaterial) insertData.tipoMaterial = input.tipoMaterial;  
  if (dimensoesLargura) insertData.dimensoesLargura = dimensoesLargura;  
  if (dimensoesAltura) insertData.dimensoesAltura = dimensoesAltura;  
  if (dimensoesComprimento) insertData.dimensoesComprimento = dimensoesComprimento;  
  
  try {
```

```
const result = await db.insert(cotacoesFrete).values(insertData);
return { success: true, id: result.insertId };<br/>
} catch (error: any) {<br/>
  console.error('INSERT ERRO:', { message: error.message, sql: error.sql, insertData });
  throw new TRPCError({ code: 'INTERNAL_SERVER_ERROR', message: error.message });
}
})
```

4. SOLUÇÃO TÉCNICA: FRONTEND E PERSISTÊNCIA

4.1 Correção do Payload (NovaCotacaoDialog.tsx)

Remover campos obsoletos e garantir a limpeza dos dados antes do envio:

1. **Remover:** `pedidoCnpj`, `pedidoEndereco`, `pedidoCep`.
2. **Remover:** O uso de `as any` no mutate para permitir validação do TypeScript.
3. **Adicionar:** Limpeza de estado no `onSuccess`.

4.2 Persistência de Aba (sessionStorage)

Para evitar a perda de dados ao alternar janelas do Chrome, implementar o hook `useEffect` para sincronizar o estado do formulário com o `sessionStorage`, garantindo que o **Dialog** recupere os valores caso o componente seja desmontado pelo navegador.

5. PERFORMANCE E FILTRAGEM (Drizzle Syntax)

Para resolver a lentidão no carregamento da página inicial, aplicar o filtro de 30 dias diretamente na query de seleção:

```
import { gte, lte, and, desc } from 'drizzle-orm';

const trintaDiasAtras = new Date();
trintaDiasAtras.setDate(trintaDiasAtras.getDate() - 30);

const solicitacoes = await db
  .select()
  .from(cotacoesFrete)
  .where(and(
    gte(cotacoesFrete.createdAt, trintaDiasAtras),
    lte(cotacoesFrete.createdAt, new Date())
  ))
  .orderBy(desc(cotacoesFrete.createdAt))
  .limit(100);
```

6. PROTOCOLO DE TESTES EXAUSTIVOS

Teste	Ação	Resultado Esperado
T1: INSERT Mínimo	Criar com apenas Nome, Cidade e UF	Sucesso no banco (campos default preenchidos)
T2: CNPJ Clean	Inserir CNPJ com pontos e barras	Banco deve registrar apenas números
T3: Persistência	Preencher e trocar de aba no Chrome	Dados devem permanecer ao voltar
T4: Performance	Abrir menu Solicitação de Frete	Carregamento em < 2 segundos

RESPONSÁVEL TÉCNICO

Local e data: Brasil, 08 de agosto de 2026

Documento elaborado em 08 de agosto de 2026. As informações contidas são de responsabilidade do solicitante.